

Kinematics for Robots and Autonomous Agents

NECKER

June 2, 2026

Contents

1	What Kinematics Is	2
2	Vectors, Matrices, and Reference Frames	2
2.1	Reference Frames (Frames)	2
2.2	Vectors and Matrices	2
2.3	Dot Product	3
2.4	Cross Product	3
3	Formal Definition of a Frame and Rotation Matrices	5
3.1	One Frame Relative to Another: The Rotation Matrix	5
4	Coordinate Transformation (Pose)	7
5	Composite Rotations	7
5.1	Elementary Rotations	7
5.2	Composition Rule	7
6	Orientation Parameterizations	7
6.1	Euler Angles (Z-Y-Z) and Roll-Pitch-Yaw	7
6.2	Quaternions	7
7	Homogeneous Transformation Matrices	9
8	Transformation Trees (TF Trees)	9
9	Summary Table of Representations	10

1 What Kinematics Is

Robots are rigid bodies that move in real three-dimensional spaces. In order for an algorithm to make actuation decisions and deliberate on movement, it requires a rigorous mathematical representation.

Kinematics is the branch of mechanics that studies the motion of bodies while ignoring the causes (forces and torques) that generate it. It focuses exclusively on the geometry of movement, describing quantities such as position, velocity, and acceleration.

2 Vectors, Matrices, and Reference Frames

2.1 Reference Frames (Frames)

The central concept of kinematics is the reference system (or *frame*). The existence of at least two fundamental frames is assumed:

- **World frame (Global System):** An absolute reference system in which all coordinates of the environment are specified.
- **Robot frame (Local System):** A frame rigidly attached to the robot. Since the robot is considered a *rigid body* (an object in which the Euclidean distance between any two points never changes), the frame moves jointly with it.

2.2 Vectors and Matrices

A **geometric vector** $\mathbf{v}$ is an entity defined by magnitude (length), direction, and sense, independent of the reference system. When associated with a frame, it becomes a coordinate vector $\mathbf{p} \in \mathbb{R}^n$, which is an ordered list of numbers.

A **matrix** is a rectangular array of scalars of size $m \times n$. The **matrix product** (non-commutative) between A ($m \times k$) and B ($k \times n$) yields C ($m \times n$) such that:

$$c_{i,j} = \sum_k a_{i,k} b_{k,j}$$

Other fundamental operations:

- **Transpose** (A^T): Obtained by swapping rows with columns.
- **Inverse** (A^{-1}): A matrix such that $AA^{-1} = I$, which exists only for square matrices with a non-zero determinant.

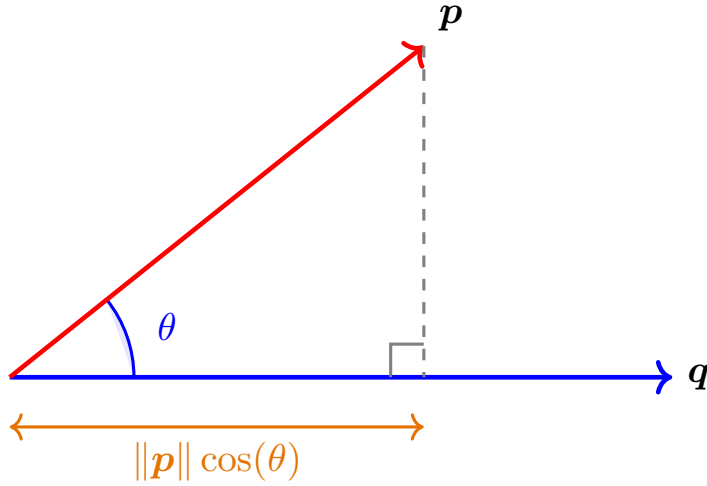
2.3 Dot Product

The scalar product between two vectors $\mathbf{p}$ and $\mathbf{q}$ returns a scalar representing the length of the projection of $\mathbf{p}$ onto $\mathbf{q}$, multiplied by the length of $\mathbf{q}$:

$$\mathbf{p} \cdot \mathbf{q} = \|\mathbf{p}\| \|\mathbf{q}\| \cos(\theta)$$

- $\mathbf{p} \cdot \mathbf{q} > 0$: directions are generally matching ($\theta < 90^\circ$).
- $\mathbf{p} \cdot \mathbf{q} = 0$: orthogonal vectors ($\theta = 90^\circ$).
- $\mathbf{p} \cdot \mathbf{q} < 0$: opposite directions ($\theta > 90^\circ$).

In matrix form: $\mathbf{p} \cdot \mathbf{q} = \mathbf{p}^T \mathbf{q} = p_x q_x + p_y q_y + p_z q_z$.



$$\mathbf{p} \cdot \mathbf{q} = \|\mathbf{p}\| \|\mathbf{q}\| \cos(\theta)$$

2.4 Cross Product

The vector product between two vectors $\mathbf{p}, \mathbf{q} \in \mathbb{R}^3$ is an operation that produces a new vector, denoted by $\mathbf{p} \times \mathbf{q}$, having the following fundamental properties:

- it is orthogonal to both $\mathbf{p}$ and $\mathbf{q}$;
- its sense is determined by the right-hand rule;
- its magnitude is equal to the area of the parallelogram generated by the two vectors.

Specifically,

$$\|\mathbf{p} \times \mathbf{q}\| = \|\mathbf{p}\| \|\mathbf{q}\| \sin \theta$$

where θ is the angle enclosed between $\mathbf{p}$ and $\mathbf{q}$. (base * height)

If the two vectors are parallel (or anti-parallel), then $\sin \theta = 0$ and therefore

$$\mathbf{p} \times \mathbf{q} = \mathbf{0}.$$

This reflects the fact that the parallelogram degenerates into a segment, having zero area.

Computationally

The cross product between two vectors $\mathbf{p}$ and $\mathbf{q}$ is computed directly component by component as:

$$\mathbf{p} \times \mathbf{q} = \begin{bmatrix} p_y q_z - p_z q_y \\ p_z q_x - p_x q_z \\ p_x q_y - p_y q_x \end{bmatrix}.$$

This is the explicit form of the cross product and derives directly from its geometric definition: the result is a vector perpendicular to $\mathbf{p}$ and $\mathbf{q}$, with a magnitude equal to the area of the parallelogram generated by them.

The same result can however be obtained in a more compact way by introducing a matrix associated with the vector $\mathbf{p}$, called the skew-symmetric matrix ($T = -T^T$):

$$[\mathbf{p}_\times] = \begin{bmatrix} 0 & -p_z & p_y \\ p_z & 0 & -p_x \\ -p_y & p_x & 0 \end{bmatrix}.$$

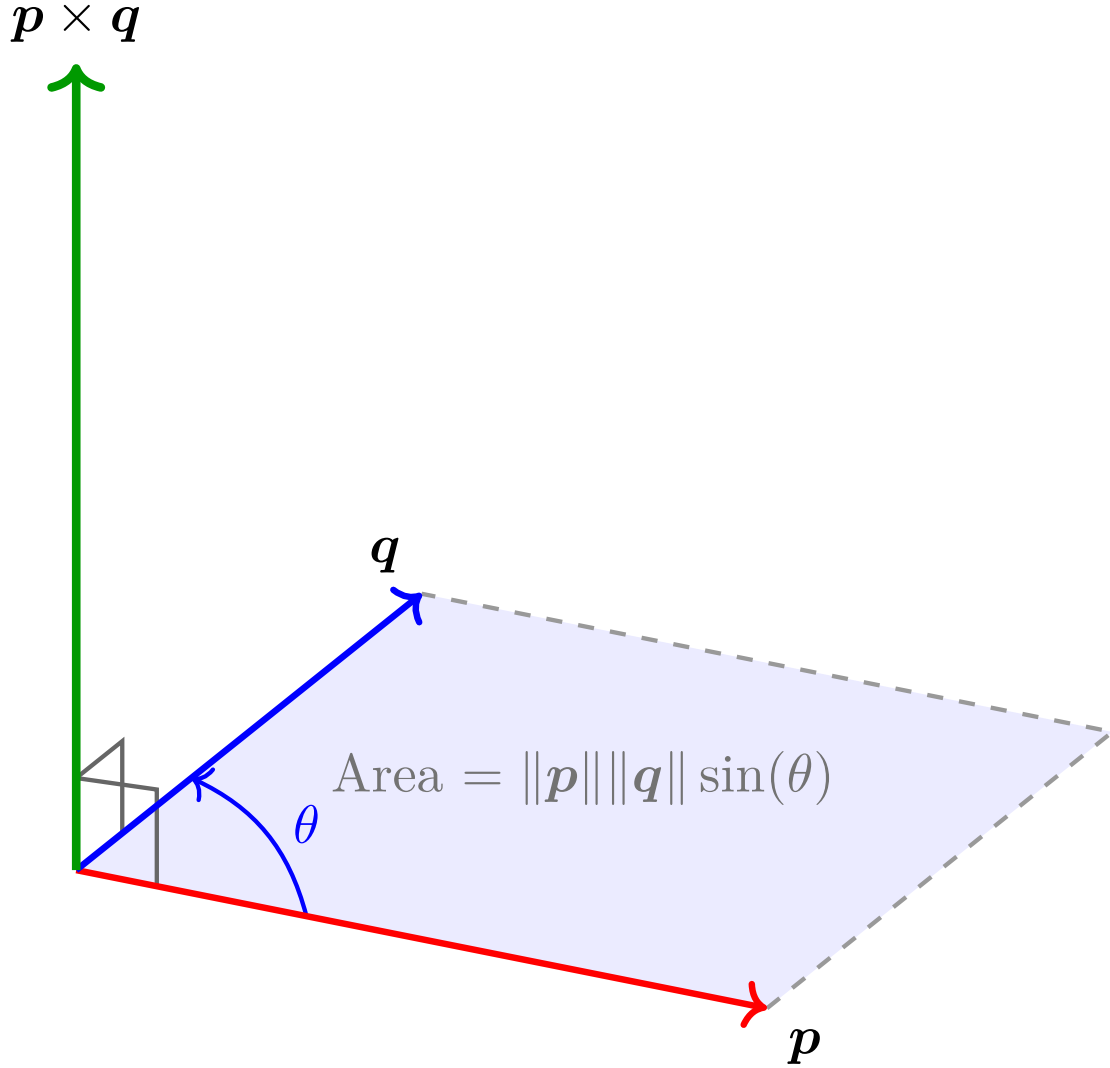
Multiplying this matrix by the vector $\mathbf{q}$ yields:

$$[\mathbf{p}_\times]\mathbf{q} = \begin{bmatrix} 0 & -p_z & p_y \\ p_z & 0 & -p_x \\ -p_y & p_x & 0 \end{bmatrix} \begin{bmatrix} q_x \\ q_y \\ q_z \end{bmatrix}.$$

Carrying out the matrix-vector product recovers exactly the same components:

$$[\mathbf{p}_\times]\mathbf{q} = \begin{bmatrix} p_y q_z - p_z q_y \\ p_z q_x - p_x q_z \\ p_x q_y - p_y q_x \end{bmatrix} = \mathbf{p} \times \mathbf{q}.$$

Therefore, the skew-symmetric matrix does not introduce a new operation, but is simply a rewriting of the cross product that allows treating it as a linear matrix-vector multiplication.



3 Formal Definition of a Frame and Rotation Matrices

Before describing movement, we must rigorously define the environment in which it takes place. In kinematics, space is described using **Reference Systems** (or *Frames*).

A frame, conventionally denoted as $O - \mathbf{x}\mathbf{y}\mathbf{z}$, is simply a mathematical framework defined by two essential elements:

1. An origin point O , which fixes the system's position in space (the root from which distance measurements start).
2. A triad of **unit vectors** (vectors of unit length) $\mathbf{x}, \mathbf{y}, \mathbf{z}$, which define the directions of the coordinate axes and, consequently, the frame's orientation.

For this triad to form a standard and physically coherent Cartesian space, the three unit vectors cannot be chosen at random, but must obey two strict geometric laws: orthonormality and the right-hand rule.

1. The Orthonormality Constraint

The axes must be perpendicular to each other (orthogonal) and the measurement scale must be unit-length (normalized). In mathematical language, using the dot product, this translates into the system:

$$\begin{cases} \mathbf{x} \cdot \mathbf{x} = 1, & \mathbf{y} \cdot \mathbf{y} = 1, & \mathbf{z} \cdot \mathbf{z} = 1 & \text{(Normality: vectors have length 1)} \\ \mathbf{x} \cdot \mathbf{y} = 0, & \mathbf{x} \cdot \mathbf{z} = 0, & \mathbf{y} \cdot \mathbf{z} = 0 & \text{(Orthogonality: axes are at } 90^\circ \text{ to each other)} \end{cases}$$

This property guarantees that the three axes represent completely independent spatial degrees of freedom: moving along x does not imply any movement along y or z .

2. The Right-Hand Rule (Space Chirality)

It is not enough for the axes to be at 90° . There are two specular ways to arrange three orthogonal axes (a right-handed one and a left-handed one). In robotics and physics, the *right-hand rule* is adopted to ensure that vector calculus (such as rotations and torques) maintains a consistent sign. This constraint is enforced through the cross product:

$$\mathbf{x} \times \mathbf{y} = \mathbf{z}, \quad \mathbf{y} \times \mathbf{z} = \mathbf{x}, \quad \mathbf{z} \times \mathbf{x} = \mathbf{y}$$

If the product $\mathbf{x} \times \mathbf{y}$ returned $-\mathbf{z}$, we would have built a left-handed system (like the mirror image of a standard system), rendering all standard kinematic conventions void.

3.1 One Frame Relative to Another: The Rotation Matrix

In reality, a robot has multiple frames (one for each joint, one for the base, one for the camera). Let us consider the problem of describing the orientation of a local frame $B = O' - \mathbf{x}'\mathbf{y}'\mathbf{z}'$ (for example, the robot's hand) seen from the "eyes" of a global frame $A = O - \mathbf{x}\mathbf{y}\mathbf{z}$ (the room).

How do we describe the inclination of B relative to A ? The strategy is simple: **we observe the three axes of B and measure them using the "meters" of frame A .**

To do this, we must take each axis of B (namely $\mathbf{x}', \mathbf{y}', \mathbf{z}'$) and project it orthogonally onto the axes of A ($\mathbf{x}, \mathbf{y}, \mathbf{z}$). The perfect mathematical tool for projecting one vector onto another is the **dot product**.

Let us take for instance the first axis of B , which is $\mathbf{x}'$. Its coordinates read in frame A will be:

$${}^A\mathbf{x}' = \begin{bmatrix} \mathbf{x}' \cdot \mathbf{x} \\ \mathbf{x}' \cdot \mathbf{y} \\ \mathbf{x}' \cdot \mathbf{z} \end{bmatrix}$$

If we repeat this procedure for all three axes of B and stack them side by side as columns, we get a single mathematical structure called the **Rotation Matrix** ${}^A R_B$:

$${}^A R_B = \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ {}^A\mathbf{x}' & {}^A\mathbf{y}' & {}^A\mathbf{z}' \\ \downarrow & \downarrow & \downarrow \end{bmatrix} = \begin{bmatrix} \mathbf{x}' \cdot \mathbf{x} & \mathbf{y}' \cdot \mathbf{x} & \mathbf{z}' \cdot \mathbf{x} \\ \mathbf{x}' \cdot \mathbf{y} & \mathbf{y}' \cdot \mathbf{y} & \mathbf{z}' \cdot \mathbf{y} \\ \mathbf{x}' \cdot \mathbf{z} & \mathbf{y}' \cdot \mathbf{z} & \mathbf{z}' \cdot \mathbf{z} \end{bmatrix}$$

The Deep Meaning: Direction Cosines

There is a fundamental detail to note: since all vectors involved ($\mathbf{x}, \mathbf{y}, \mathbf{z}$ and $\mathbf{x}', \mathbf{y}', \mathbf{z}'$) are *unit vectors* (they have length 1), their dot product simplifies enormously. Recalling the trigonometric definition of the dot product ($\mathbf{v} \cdot \mathbf{w} = \|\mathbf{v}\| \|\mathbf{w}\| \cos \theta$), we find that:

$$\mathbf{x}' \cdot \mathbf{x} = (1)(1) \cos(\theta_{\mathbf{x}'\mathbf{x}}) = \cos(\theta_{\mathbf{x}'\mathbf{x}})$$

Every single element of the rotation matrix is nothing more than the **cosine of the angle** formed between an axis of frame B and an axis of frame A .

NOTE: ultimately a rotation consists in finding where the tip of the vector is located (starting with the vectors of the three axes which then move due to the rotation), so the x and y components (found with sine and cosine) are used to identify the new position

graphically: <https://www.youtube.com/watch?v=-pQtKHEoueE>

4 Coordinate Transformation (Pose)

The *Pose* of a robot is the union of its position (origin ${}^A O'$) and its orientation (${}^A R_B$). Given a point ${}^B \mathbf{p}$ in frame B , to find its coordinates in the global frame A , we add the translation to the rotated vector:

$${}^A \mathbf{p} = {}^A O' + {}^A R_B {}^B \mathbf{p}$$

5 Composite Rotations

5.1 Elementary Rotations

Rotations around the principal axes by an angle α are described by:

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix}, \quad R_y(\alpha) = \begin{bmatrix} \cos \alpha & 0 & \sin \alpha \\ 0 & 1 & 0 \\ -\sin \alpha & 0 & \cos \alpha \end{bmatrix}, \quad R_z(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

5.2 Composition Rule

When applying rotations in sequence $R_1, R_2, \dots, R_n$:

- If the rotations take place with respect to the **fixed axes** (global), one **pre-multiplies**: ${}^A R_B = R_n \dots R_2 R_1$.
- If the rotations take place with respect to the **current axes** (local), one **post-multiplies**: ${}^A R_B = R_1 R_2 \dots R_n$.

Furthermore, concatenated matrices allow cancellations of indices/superscripts: ${}^A R_B {}^B R_C = {}^A R_C$.

6 Orientation Parameterizations

Rotation matrices use 9 parameters to describe 3 degrees of freedom. They are *redundant*. Minimal representations exist:

6.1 Euler Angles (Z-Y-Z) and Roll-Pitch-Yaw

- **Roll-Pitch-Yaw**: Rotations respectively on X , Y , and Z of the **fixed axes**.
- **Z-Y-Z Euler Angles**: Three successive rotations around the **current axes**.

The main problem with 3-angle parameterizations is **Singularities (Gimbal Lock)**. For instance, in a Z-Y-Z configuration, if the intermediate angle $\beta = \pi$ or $\beta = 0$, the first and last axes of rotation become aligned. This causes the loss of one degree of freedom (the system "collapses") and prevents computing the inverse problem. It is analogous to being at the Earth's North Pole: longitude loses its mathematical meaning.

6.2 Quaternions

To overcome the limitations of rotation matrices and Euler angles — particularly the problem of *Gimbal Lock* — we resort to using **Quaternions**. Introduced by the mathematician William Rowan Hamilton, they represent an extension of complex numbers into a four-dimensional space.

A quaternion $\mathbf{q} \in \mathbb{H}$ is defined as a linear combination of a real scalar and three imaginary units i, j, k :

$$\mathbf{q} = q_0 + q_1 i + q_2 j + q_3 k$$

where the coefficients $q_0, q_1, q_2, q_3 \in \mathbb{R}$, and the imaginary units satisfy the fundamental relations:

$$i^2 = j^2 = k^2 = ijk = -1$$

Often, it is useful to express the quaternion by separating its scalar part from its vector part: $\mathbf{q} = (q_0, \mathbf{v})$, with $\mathbf{v} = [q_1 \ q_2 \ q_3]^T$.

To describe rotations in three-dimensional space, we exclusively use **Unit Quaternions** (meaning with norm $\|\mathbf{q}\| = 1$). According to **Euler's Rotation Theorem**, any 3D rotation can be described by an angle θ

around an arbitrary rotation axis, identified by a unit vector $\mathbf{r} = [r_x, r_y, r_z]^T$. The unit quaternion associated with this rotation is formulated as:

$$\mathbf{q} = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)(r_x i + r_y j + r_z k)$$

The use of the half-angle $\frac{\theta}{2}$ stems from the nature of the rotation operator. To apply the rotation to a point or a vector $\mathbf{p}$ in space (represented as a pure quaternion $\mathbf{p} = 0 + p_x i + p_y j + p_z k$), we use the quaternion product via the conjugate:

$$\mathbf{p}' = \mathbf{q} \otimes \mathbf{p} \otimes \mathbf{q}^*$$

where $\mathbf{q}^*$ is the quaternion conjugate ($\mathbf{q}^* = q_0 - \mathbf{v}$) and $\otimes$ denotes the Hamilton product. The double multiplication applies the rotation of $\frac{\theta}{2}$ twice, resulting in a net rotation of exactly θ .

Practical Rotation Example:

Consider wanting to rotate a vector $\mathbf{v}_{in} = [0 \ 1 \ 0]^T$ (located along the y -axis) by an angle $\theta = 90^\circ$ ($\frac{\pi}{2}$ radians) around the z -axis, identified by the unit vector $\mathbf{r} = [0 \ 0 \ 1]^T$. Geometrically, the rotated vector is expected to lie on the negative x -axis, meaning $\mathbf{v}_{out} = [-1 \ 0 \ 0]^T$.

1. **Construction of the rotation quaternion $\mathbf{q}$:** Since $\frac{\theta}{2} = 45^\circ$ ($\frac{\pi}{4}$ rad), we have $\cos(\frac{\pi}{4}) = \frac{\sqrt{2}}{2}$ and $\sin(\frac{\pi}{4}) = \frac{\sqrt{2}}{2}$. Therefore:

$$\mathbf{q} = \frac{\sqrt{2}}{2} + 0i + 0j + \frac{\sqrt{2}}{2}k = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k$$

Its conjugate $\mathbf{q}^*$ is:

$$\mathbf{q}^* = \frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}k$$

2. Encoding the vector as a pure quaternion $\mathbf{p}$:

$$\mathbf{p} = 0 + 0i + 1j + 0k = j$$

3. **Calculation of the first product $\mathbf{q} \otimes \mathbf{p}$:** Exploiting distributive rules and knowing that $k \otimes j = -i$, we get:

$$\mathbf{q} \otimes \mathbf{p} = \left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k\right) \otimes j = \frac{\sqrt{2}}{2}j + \frac{\sqrt{2}}{2}(k \otimes j) = -\frac{\sqrt{2}}{2}i + \frac{\sqrt{2}}{2}j$$

4. **Calculation of the final product $\mathbf{p}' = (\mathbf{q} \otimes \mathbf{p}) \otimes \mathbf{q}^*$:** Multiplying the previous result by the conjugate $\mathbf{q}^*$ and recalling product identities ($i \otimes k = -j$ and $j \otimes k = i$), we expand the terms:

$$\mathbf{p}' = \left(-\frac{\sqrt{2}}{2}i + \frac{\sqrt{2}}{2}j\right) \otimes \left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}k\right)$$

$$\mathbf{p}' = -\frac{2}{4}i + \frac{2}{4}(i \otimes k) + \frac{2}{4}j - \frac{2}{4}(j \otimes k)$$

$$\mathbf{p}' = -\frac{1}{2}i + \frac{1}{2}(-j) + \frac{1}{2}j - \frac{1}{2}(i)$$

$$\mathbf{p}' = -\frac{1}{2}i - \frac{1}{2}j + \frac{1}{2}j - \frac{1}{2}i = -1i$$

The resulting quaternion is $\mathbf{p}' = 0 - 1i + 0j + 0k$. Extracting the vector part yields exactly the sought vector: $\mathbf{v}_{out} = [-1 \ 0 \ 0]^T$.

7 Homogeneous Transformation Matrices

In three-dimensional Euclidean space, a linear transformation (such as a rotation) can be expressed via a 3×3 matrix. However, translation is an *affine* operation, which would require a separate vector addition ($\mathbf{p}' = R\mathbf{p} + \mathbf{d}$). To bypass this and handle pose (rotation) and translation with a single, more efficient multiplicative operation, we project the 3D space into a 4-dimensional space via **Homogeneous Coordinates**.

We add a fourth coordinate, called "weight" or w , to three-dimensional vectors, which are divided into two categories depending on **the value of the fourth coordinate**:

- **Points in space:** $\mathbf{p} = [p_x, p_y, p_z, 1]^T$. The value "1" indicates that the point possesses a defined position and will undergo the effect of any translations.
- **Directions (or pure vectors):** $\mathbf{v} = [v_x, v_y, v_z, 0]^T$. The value "0" "turns off" the translation component of the matrix, making the vector immune to shifts, for example when the vector represents a Direction or a pure vector (e.g., a force, a velocity).

The 4×4 **Homogeneous Transformation Matrix** ${}^A T_B$ merges into a single entity the rotation matrix ${}^A R_B \in SO(3)$ (which describes the orientation of frame B relative to A) and the translation vector ${}^A O'_B \in \mathbb{R}^3$ (which describes the origin of frame B relative to A):

$${}^A T_B = \begin{bmatrix} r_{11} & r_{12} & r_{13} & d_x \\ r_{21} & r_{22} & r_{23} & d_y \\ r_{31} & r_{32} & r_{33} & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} {}^A R_B & {}^A O'_B \\ \mathbf{0}^T & 1 \end{bmatrix}$$

Practical Example and Fast Inversion:

Let's imagine having a 3D scanner mounted on the end-effector of a robotic arm. The sensor detects a point on a mechanical part: $\mathbf{p}_{cam} = [0, 0, 0.5, 1]^T$ (half a meter along the optical z -axis). We know that the camera frame relative to the robot base is translated by $0.5m$ along x and rotated by 90° ($\pi/2$) around the y -axis. The matrix mapping points from the camera to the base will be:

$${}^{base} T_{cam} = \begin{bmatrix} 0 & 0 & 1 & 0.5 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Multiplying ${}^{base} T_{cam} \cdot \mathbf{p}_{cam}$, we obtain the coordinates of the point read from the robot base reference system: $\mathbf{p}_{base} = [1, 0, 0, 1]^T$. This matrix approach is computationally ideal for transforming entire *Point Clouds* in one single pass by multiplying the 4×4 matrix by a $4 \times N$ matrix containing all the scanned points.

A crucial advantage of homogeneous matrices is the efficiency in computing the **inverse matrix** T^{-1} (which maps from A to B). Instead of performing an expensive Gauss-Jordan inversion on a 4×4 , we exploit the orthogonal properties of the rotation submatrix ($R^{-1} = R^T$):

$$({}^A T_B)^{-1} = {}^B T_A = \begin{bmatrix} {}^A R_B^T & -{}^A R_B^T \cdot {}^A O'_B \\ \mathbf{0}^T & 1 \end{bmatrix}$$

8 Transformation Trees (TF Trees)

A real robotic system is almost never composed of only two reference systems. A manipulator on a mobile base equipped with vision sensors involves dozens of frames (e.g., `odom`, `base_link`, `link_1` ... `link_6`, `camera_link`, `object_frame`). Maintaining and manually calculating every possible spatial relationship between arbitrary pairs of frames requires a computational burden of $O(N^2)$ and is impossible to manage dynamically.

In modern architectures like ROS 2 (the `tf2` library), this problem is solved by structuring frames into a **TF Tree**, which is a Directed Acyclic Graph (DAG).

- **Nodes:** Represent individual physical or logical coordinate *frames* of the robot.
- **Edges (Archi):** Represent the homogeneous transformation matrix ${}^{parent} T_{child}$ that describes the pose of the "child" node relative to the "parent" node. A valid tree dictates that every node has exactly one parent, guaranteeing a single global origin (often `world` or `map`).

Kinematic Resolution on a TF Tree:

When a computation node (e.g., MoveIt) needs to know the transformation between two distant frames in the tree, for example from `base_link` (B) to `camera_link` (C), the system performs a breadth-first or depth-first search to find the unique path connecting them.

1. **Path search:** The connecting branch is identified, passing through intermediate nodes (e.g., the arm joints: $B \rightarrow L_1 \rightarrow L_2 \rightarrow \dots \rightarrow L_6 \rightarrow C$).
2. **Concatenation (Chain Rule):** The homogeneous transformation matrices are multiplied in sequence along the path. The multiplication order (from right to left for transformations in current frames) is essential:

$${}^B T_C = {}^B T_{L_1} \cdot {}^{L_1} T_{L_2} \dots {}^{L_6} T_C$$

3. **Descending the tree (Parent $\rightarrow$ Child):** If the path goes down an edge (the graph's natural direction), the matrix stored in memory ${}^P T_F$ is used directly.
4. **Ascending the tree (Child $\rightarrow$ Parent):** If the path requires going up a branch before going down another (e.g., from `camera_link` to `gripper_link` passing through `link_6`), the algorithm inverts the matrix of the ascended edge on the fly using the fast inverse formula: $({}^P T_F)^{-1} = {}^F T_P$.

This approach decouples data publishing: sensors only update the leaf branches at high frequency (e.g., `camera` $\rightarrow$ `object`), motor encoders update internal kinematics, and the navigation algorithm updates the root, guaranteeing a coherent and scalable snapshot of the entire robot at every instant t .

9 Summary Table of Representations

Method	Advantages (Pros)	Disadvantages (Cons)	Typical Use (ROS)
Rot. Matrices (3x3)	No singularities, unique, simple math	Redundant (9 vars), unstable if modified	Internal calculations
Euler Angles	Minimal (3 vars), intuitive	Depend on the order, singularities	URDF, user interfaces
Quaternions (4D)	Efficient, no singularities, great for SLERP	Complex math, less intuitive	ROS Messages
TF Matrices (4x4)	Unified representation, fast computation	Redundant	Internal TF lookups